

Laboratorium 3: Łączenie tabel i funkcje agregujące.

Cel laboratorium

Celem zajęć jest opanowanie umiejętności łączenia danych z wielu tabel za pomocą operatora `INNER JOIN` oraz wykonywania obliczeń na grupach rekordów przy użyciu funkcji agregujących (`COUNT` , `SUM` , `AVG` , `MIN` , `MAX`) wraz z klauzulami `GROUP BY` i `HAVING` .

Podstawy teoretyczne

1. Łączenie tabel (INNER JOIN)

Łączenie wewnętrzne pozwala na zestawienie wierszy z dwóch lub więcej tabel na podstawie wspólnej kolumny (zazwyczaj klucza obcego i klucza głównego).

```
SELECT kolumny
FROM TabelaA
INNER JOIN TabelaB ON TabelaA.klucz_obcy = TabelaB.id;
```

2. Funkcje agregujące

Służą do wykonywania obliczeń na zbiorze wartości i zwracania pojedynczego wyniku:

- `COUNT( *)` – liczy liczbę wierszy.
- `SUM(kolumna)` – sumuje wartości w kolumnie.
- `AVG(kolumna)` – oblicza średnią wartość.
- `MIN(kolumna)` / `MAX(kolumna)` – znajduje wartość minimalną lub maksymalną.

3. Grupowanie danych (GROUP BY i HAVING)

- `GROUP BY` – dzieli wynik zapytania na grupy w celu zastosowania funkcji agregujących dla każdej grupy.

- **HAVING** – służy do filtrowania grup (działa jak **WHERE** , ale dla wyników funkcji agregujących).

Przygotowanie środowiska

Zadania należy wykonywać na własnej bazie danych. Przed przystąpieniem do zadań należy zaimportować strukturę bazy danych i przykładowe dane dotyczące ligi piłkarskiej: 🖱️ [Skrypt SQL: lab03_football_league.sql](#)

Schemat bazy danych (Mermaid)

```
erDiagram
    Stadiony ||--o{ Druzyiny : "jest siedzibą"
    Stadiony ||--o{ Mecze : "gości"
    Druzyiny ||--o{ Pilkarze : "zatrudnia"
    Druzyiny ||--o{ Mecze : "gra jako gospodarz"
    Druzyiny ||--o{ Mecze : "gra jako gość"

    Stadiony {
        int id PK
        string nazwa
        string miasto
        int pojemnosc
    }
    Druzyiny {
        int id PK
        string nazwa
        string miasto
        int rok_zalozenia
        int id_stadionu FK
    }
    Pilkarze {
        int id PK
        string imie
        string nazwisko
        string pozycja
        numeric wynagrodzenie
        int id_druzyiny FK
    }
    Mecze {
        int id PK
```

```

        int id_gospodarza FK
        int id_goscia FK
        int id_stadionu FK
        date data_meczu
        int bramki_gospodarza
        int bramki_goscia
    }

```

Przykłady startowe: Łączenie i agregacja

Jak działa INNER JOIN ?

Łącząc tabele Piłkarze i Drużyny , SQL dopasowuje id_drużyny z tabeli Piłkarze do id z tabeli Drużyny .

```

SELECT Piłkarze.nazwisko, Drużyny.nazwa
FROM Piłkarze
INNER JOIN Drużyny ON Piłkarze.id_drużyny = Drużyny.id;

```

Wizualizacja złączenia:

```

graph LR
    P[Piłkarze] -- id_drużyny --> D[Drużyny]
    D -- id --> P
    subgraph Wynik
        R[Nazwisko + Nazwa Drużyny]
    end
end

```

Jak działa GROUP BY ?

Grupowanie pozwala na policzenie rekordów (np. ilu piłkarzy gra w każdej drużynie).

```

SELECT id_drużyny, COUNT(*) AS liczba_piłkarzy
FROM Piłkarze
GROUP BY id_drużyny;

```

Wizualizacja grupowania:

```
graph TD
    A[Dane wejściowe] --> B{Grupuj wg id_drużyny}
    B --> C1[Grupa: Drużyna 1]
    B --> C2[Grupa: Drużyna 2]
    C1 --> D1[Agregacja: COUNT]
    C2 --> D2[Agregacja: COUNT]
    D1 --> E[Wynik zbiorczy]
    D2 --> E
```

Jak działa HAVING ?

Klauzula `HAVING` służy do filtrowania grup danych po wykonaniu agregacji. Różni się od `WHERE` tym, że może operować na wynikach funkcji takich jak `COUNT` czy `SUM`.

```
SELECT id_drużyny, COUNT(*) AS liczba_pilkarzy
FROM Pilkarze
GROUP BY id_drużyny
HAVING COUNT(*) > 1; -- Filtrowanie tylko tych grup, które mają więcej i
```

Wizualizacja filtrowania grup:

```
graph TD
    A[Wszystkie Grupy] --> B{Liczba > 1?}
    B -- Tak --> C[Wyświetl grupę]
    B -- Nie --> D[Odrzuć grupę]
```

Łączenie wielu tabel

W SQL możemy łączyć więcej niż dwie tabele jednocześnie, aby uzyskać pełniejszy obraz danych.

```
-- Wyświetl zawodnika, jego drużynę oraz miasto, w którym znajduje się :
SELECT P.nazwisko, D.nazwa AS nazwa_drużyny, S.miasto
FROM Pilkarze P
INNER JOIN Drużyny D ON P.id_drużyny = D.id
```

INNER JOIN Stadiony S ON D.id_stadionu = S.id;

Wizualizacja połączenia wielokrotnego:

```
graph LR
    P[Piłkarze] -- id_drużyny --> D[Drużyny]
    D -- id_stadionu --> S[Stadiony]
    subgraph Połączone_Dane
        R[Nazwisko + Drużyna + Miasto]
    end
    end
```

1. Wyświetl listę piłkarzy wraz z nazwą ich drużyny.
2. Wyświetl wszystkie mecze, podając nazwy drużyn gospodarzy i gości zamiast ich identyfikatorów.
3. Znajdź wszystkie stadiony, na których swoje mecze rozgrywa drużyna 'Legia Warszawa' (użyj złączenia tabel Drużyny i Stadiony).
4. Policz, ilu piłkarzy gra w każdej z drużyn. Wynik powinien zawierać nazwę drużyny i liczbę graczy.
5. Oblicz średnie wynagrodzenie piłkarzy w całej lidze.
6. Znajdź maksymalne i minimalne wynagrodzenie w drużynie 'Lech Poznań'.
7. Wyświetl nazwy drużyn wraz z łączną sumą wynagrodzeń ich piłkarzy, posortowaną malejąco.
8. Wyświetl listę stadionów wraz z liczbą meczów, które się na nich odbyły.
9. Oblicz łączną liczbę bramek strzelonych przez gospodarzy we wszystkich meczach.
10. Wyświetl nazwy drużyn, które mają więcej niż 1 piłkarza w kadrze (użyj GROUP BY i HAVING).
11. Wyświetl mecze (data, nazwa gospodarza, nazwa gościa), które odbyły się na stadionach o pojemności powyżej 30 000 miejsc.
12. Znajdź średnią liczbę bramek strzelonych przez gości w meczach rozegranych we wrześniu 2023 roku.
13. Wyświetl imiona i nazwiska piłkarzy grających na pozycji 'Napastnik' wraz z nazwą ich drużyny.
14. Znajdź drużyny założone przed 1920 rokiem i policz ich piłkarzy.

15. Wyświetl nazwę stadionu i jego miasto dla wszystkich meczów, w których padł remis (bramki gospodarza = bramki gościa).